

RAG & Embeddings

1. Why LLMs Alone Are Not Enough

Core Limitation of LLMs

Large Language Models (LLMs) such as GPT-4, Claude, Gemini, etc., are extremely powerful at generating fluent and coherent text. However, they have a fundamental limitation:

LLMs cannot see your private data.

Key reasons:

- They are trained primarily on **public data** available on the internet up to a certain cutoff date.
- They have **no access** to company documents, internal knowledge bases, private databases, or real-time proprietary information.
- When asked about internal or private information, they often **hallucinate** (make up confident but incorrect answers).

The “Brain in a Windowless Room” Problem

This is a powerful analogy used to explain the limitation:

- Imagine a brilliant person locked inside a windowless room filled only with books that were printed before a certain date.
- The person has excellent knowledge of everything inside those books.
- But they have **no way** to see new information, private company documents, or the outside world.
- Their knowledge is frozen in time and confined to the fixed training data.

This is exactly how a pure LLM works. Its knowledge is confined to the fixed dataset it was trained on. It cannot break out of those pre-trained boundaries.

Result: Without external knowledge injection, LLMs are unreliable for enterprise or domain-specific use cases that require accurate, up-to-date, or private information.

2. What is RAG?

RAG = Retrieval Augmented Generation

RAG is a technique that gives LLMs access to external knowledge (especially private and real-time data) so they can generate more accurate and grounded answers.

Three Core Components of RAG:

Component	Meaning	What it does
Retrieval	Find relevant information from external data sources	Searches your documents / knowledge base
Augmented	Enhance the LLM's prompt with the retrieved knowledge	Injects the relevant context into the prompt
Generation	Produce accurate answers based on the complete context	LLM generates the final answer using both its knowledge + retrieved data

One-line definition:

RAG provides LLMs with access to internal and real-time private documents to reduce hallucinations and provide accurate, grounded information.

3. How RAG Works (4 Simple Steps)

1. **User Asks a Question**

The user submits a natural language query.

2. **Retrieve Documents**

The system searches a knowledge base (vector store) and finds the most relevant pieces of information (chunks).

3. **Augment Prompt**

The retrieved relevant chunks are inserted into the prompt that will be sent to the LLM.

4. **Generate Answer**

The LLM receives both the original question + the relevant context and produces a grounded answer.

Detailed RAG Architecture (6 Steps)

A more complete pipeline looks like this:

- 1. **User Query** → Raw question from the user
- 2. **Embedding** → Convert the query into a numerical vector (embedding)
- 3. **Vector Search** → Search the vector database for similar document chunks
- 4. **Top-K Chunks** → Select the most relevant K chunks (commonly K=3 to K=10)

5. **LLM** → Send the original query + selected chunks to the LLM
6. **Grounded Answer** → LLM generates a response that is grounded in the retrieved information

This architecture is the foundation of almost every modern enterprise RAG system.

4. Real-World Use Cases of RAG

RAG is extremely useful whenever an organization has private knowledge that should not be baked into the LLM's training data.

Common high-value use cases:

Use Case	Example Documents / Data
Internal Docs	Company wikis, process documents, SOPs
HR Policies	Leave policy, code of conduct, benefits
Product Manuals	Technical documentation, user guides
Financial Reports	Quarterly reports, internal financial data
Customer Support	Past tickets, knowledge base articles
Legal Contracts	Agreements, NDAs, compliance documents

Other popular applications:

- Employee onboarding assistants
 - Sales enablement bots
 - Research assistants over proprietary papers
 - Codebase Q&A systems
 - Medical / legal document analysis (with proper compliance)
-

5. What is an Embedding?

An **embedding** is a technique for representing discrete items (words, sentences, documents, images, users, etc.) as **dense vectors** of real numbers in a continuous high-dimensional space.

Key Idea

- Instead of treating text as discrete tokens, we convert meaning into a list of numbers.

- Example: The phrase “Machine Learning” might become a vector such as $[0.23, -0.45, 0.67, \dots]$

These vectors are learned during model training so that **semantically similar items end up close to each other** in the vector space.

Why this is powerful:

Computers understand numbers extremely well. Once meaning is converted into vectors, we can use mathematical operations (distance, similarity, clustering, etc.) to work with language.

6. Why Vectors? (Semantic GPS)

Embeddings give computers a **Semantic GPS**.

- Words, sentences, and concepts are mapped into a multi-dimensional mathematical space.
- Vectors act as precise coordinates.
- This allows computers to understand **semantic relationships** rather than just keyword matches.

Example:

- “King – Man + Woman $\approx$ Queen” (classic vector arithmetic)
- Documents about “neural networks” will be close to documents about “deep learning”

This is the foundation of modern semantic search.

7. How Do We Measure Similarity?

Once we have vectors, we need ways to measure how similar two vectors are.

Common Similarity / Distance Metrics

Metric	Formula / Idea	Characteristics	When to Use
Cosine Similarity	$\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (\ \mathbf{A}\ \ \mathbf{B}\)$	Measures angle between vectors (direction). Ignores magnitude. Range: -1 to +1	Most popular for text embeddings
Dot Product	$\mathbf{A} \cdot \mathbf{B} = \sum (a_i \times b_i)$	Combines magnitude and direction	When magnitude carries meaning
Euclidean Distance	$d = \sqrt{\sum (a_i - c_i)^2}$	Straight-line physical distance	When absolute distance matters

Practical Note:

In most RAG systems, **Cosine Similarity** (or its normalized variants) is the default choice because we care more about semantic direction than absolute magnitude.

8. Popular Embedding Models

Model	Dimensions	Strengths	Best For
OpenAI text-embedding-3	Flexible (e.g. 3072, user-defined)	Extremely high semantic accuracy, adaptable, well-documented API	Large-scale, accurate text retrieval
Cohere	1024, 4096	Optimized for business & enterprise NLP, highly multilingual	Chatbots, global business applications
SentenceTransformers	Varies (e.g. 384, 768)	Open-source, highly customizable, efficient, great community	Local deployment, experimentation, specific tasks
Titan Multimodal	Varies (multi-modal)	Integrates visual + textual embeddings	Cross-modal search (text → image), product catalogs

Important:

Always choose the embedding model based on your use case (language support, latency, cost, accuracy, open-source requirement).

9. Dimensions vs Cost Trade-off

Embedding dimension (vector size) is a critical design decision.

High Dimensions (e.g. 1536, 3072)	Low Dimensions (e.g. 384, 512)
Higher semantic accuracy	Lower latency
Better retrieval quality	Reduced computing & storage cost
More expensive to store & search	Faster inference

Common practical choices:

- 1. **384** → Efficient & Fast (great for many production systems)
- 2. **1024** → Balanced performance
- 3. **1536** → Maximum precision (OpenAI’s older default)

Rule of thumb:

Start with a strong mid-sized model (768–1024) and only go higher if retrieval quality is insufficient.

10. Key Takeaways

1. RAG Enhancement

RAG integrates external data to optimize LLM performance and ensure current, private context is available.

2. Embedding Vitality

Embeddings are critical for robust semantic understanding and accurate knowledge retrieval.

3. Strategic Trade-offs

Dimension selection (e.g., 384 vs 1536) must carefully balance semantic accuracy against speed and computing cost.

4. Multimodal Unification

Unified embeddings enable powerful cross-modal capabilities such as text-to-image search and product catalog retrieval.

Summary

- Pure LLMs are “brains in a windowless room.”
- RAG opens a window by retrieving relevant private data and feeding it to the LLM.
- Embeddings convert meaning into numbers so computers can perform semantic search.
- Similarity metrics (especially cosine similarity) decide which documents are relevant.
- Choosing the right embedding model and dimension is a practical engineering decision involving accuracy vs cost/latency trade-offs.